Product Selection Guide Block

1. Overview

The **Product Selection Guide component** integrates with an external Feature Collection service

.

Component acts as a lightweight wrapper that orchestrates parameter extraction and HTTP communication with the external service, returning the pre-rendered HTML response directly into the page.

T Nalgene Bottle, Carboy and Vial Selection Guide | Thermo Fisher Scientific - US

2. Current AEM Implementation

- Authors configure the component using **featureCollectionId** (mandatory)
- The component relies on the **featureCollectionId** to fetch and render the appropriate product selection guide.
- At runtime, the component uses a **Sling Model (server-side)** to:
- Read authored properties
- Extract request parameters (filters, paging, etc.)
- Derive contextual information such as:
  - locale (language/country)
  - device type (mobile vs desktop)

The Sling Model then:

1. Constructs a request to the external **Feature Collection service**
2. Passes:
   - featureCollectionId
   - query parameters
   - headers/cookies (locale, user-agent, etc.)
3. Performs a **server-side HTTP call**
4. Receives a response in the form of **HTML (including CSS and JS)**
5. Directly injects this HTML into the AEM page

3. Key Concern

The current implementation relies on **HTML passthrough from an external service**, which introduces significant concerns:

- External HTML may include **uncontrolled CSS and JavaScript**
- Increased risk of:
  - render-blocking resources
  - layout shifts
  - degraded Core Web Vitals (LCP, TBT)(Largest Contentful Paint, Total Blocking Time)
- Limited control over markup, accessibility, and styling
- Tight coupling between backend response and frontend rendering.

This pattern is **not suitable for EDS**, where performance, control, and predictable rendering are critical.

4. Proposed Approach – JSON + EDS Block

Replace HTML passthrough with a **structured JSON contract**, and let the **EDS block handle rendering (decoration)**.

- **Backend → Provides data (JSON)**
- **EDS Block → Handles rendering (HTML, CSS, JS)**

EDS Block Behavior

The **product-selection-guide block** will:

- Read authored configuration:
  - featureCollectionId
  - optional display settings
- Read runtime context from the URL:
  - filters
  - paging
  - view type
- Determine locale using EDS-supported mechanisms (URL structure or metadata)
- Call the **Feature Collection service (JSON endpoint)**
- Render:
  - product listing (grid/list)
  - filters and controls
  - pagination
  - empty and error states

- The external Feature Collection service should expose a **structured JSON API** instead of returning pre-rendered HTML.

This establishes a clear separation of responsibilities:

- **Backend → Business logic and data**
- **EDS Block → Rendering and interaction**

Adopting a JSON-based contract is critical because:

- Prevents uncontrolled injection of external CSS and JavaScript
- Ensures **full control over markup, styling, and behavior within EDS**
- Enables optimized rendering, lazy loading, and better Core Web Vitals
- Improves maintainability by decoupling backend output from frontend structure

EDS blocks should **consume structured data, not pre-rendered HTML**.

5. Pros and Cons

Aspect	Current AEM (HTML Passthrough)	Proposed (JSON + EDS Block)
Performance (Core Web Vitals)	Risky due to external CSS/JS injection	Strong control with optimized rendering
Control over UI	Low (backend defines HTML)	High (EDS controls DOM and styling)
Maintainability	Tightly coupled to backend output	Clean separation of data and presentation
Authoring Experience	Simple, based on featureCollectionId	Same model retained
Flexibility	Limited	High (UI changes independent of backend)
Accessibility & Semantics	Depends on external HTML	Fully controllable in EDS
EDS Alignment	Not aligned	Fully aligned with EDS principles

6. Summary

The current implementation uses a **server-side HTML passthrough model**, where AEM acts as a proxy to an external service and renders its response directly. While functional, this introduces performance and maintainability challenges.

The recommended approach is to adopt a **JSON-driven integration with an EDS block**, where:

- The backend provides structured data
- The EDS block handles rendering and interaction

This results in:

- Improved performance and Core Web Vitals
- Full control over markup and styling
- A scalable, maintainable architecture aligned with EDS best practices

Ownership & Integration Boundaries

Ownership Matrix

Component	Owner	Responsibility
Product Selection Guide Block (EDS)	Adobe / Implementation Team	Block JS/CSS, rendering (grid/list/filters/pagination/empty/error states), URL parameter handling, locale detection, UX interactions
Feature Collection Service (JSON API)	TFS	Exposes structured JSON endpoint, business logic, filtering, sorting, pagination logic, product data
JSON API Contract	Joint (Adobe + TFS)	Request/response schema agreed during implementation
API Authentication (if needed)	TFS provides credentials; Adobe configures (in Edge Worker if API can't be accessed from browser)	Depends on API authentication.

Key Dependency: TFS Must Deliver JSON API

The EDS integration **requires TFS to expose a structured JSON endpoint** from the Feature Collection service. The current HTML passthrough endpoint cannot be used in EDS.